

Red (autore: Vittorio Albertoni)

Premessa

Red, come si capirà meglio leggendo queste pagine, è, per molti aspetti, un linguaggio rivoluzionario.

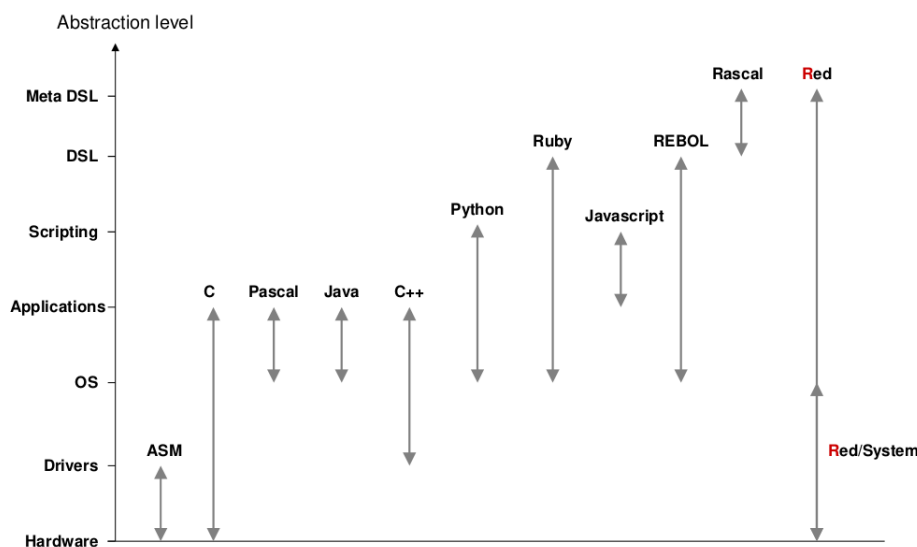
Occorre però riconoscere che di questa caratteristica esso è debitore ad un altro linguaggio al quale si ispira, il linguaggio REBOL, acronimo di Relative Expression Based Object Language. Questo linguaggio, creato dal veterano della programmazione per personal computers Carl Sassenrath, primo sviluppatore di AmigaOS, ha un nome la cui pronuncia inglese richiama il termine «ribelle» e, a detta del suo creatore, esso infatti si ribella al concetto che il software moderno debba essere grande e complesso.

In REBOL, creato verso la fine del secolo scorso, tutto è all'insegna del minimalismo: risibile impiego di risorse, file di dimensioni minime, eliminazione della verbosità che caratterizza più o meno tutti i linguaggi di programmazione.

REBOL si è affermato come domain-specific language (in sigla DSL), linguaggio di dominio specifico nello sviluppo software e nell'ingegneria di dominio.

Basandosi su questo linguaggio Nenad "DocKimbel" Rakočević ha creato il linguaggio Red, che vuole essere un linguaggio all purpose.

Il suo stesso creatore, in una conferenza tenutasi nel 2013, dopo un paio d'anni dall'avvio dei lavori per il nuovo linguaggio, così collocava il suo futuro linguaggio, con una lodevole ambizione, tra i principali linguaggi esistenti



E dopo molti anni di lavoro e di continui affinamenti molto probabilmente l'obiettivo sta per essere raggiunto.

Come si vede dal grafico, il linguaggio si presenta in due edizioni: Red e Red/System.

Lasciando Red/System ai professionisti che si occupano di software di sistema, noi qui ci occuperemo di Red e scopriremo che è un linguaggio adatto anche per principianti e dilettanti e sa essere anche molto divertente.

Di Red/System utilizzeremo solo il compilatore per il codice Red.

Indice

1	Predisposizione del computer	3
2	Come funziona	3
3	Il linguaggio	6
3.1	Parole	6
3.2	Tipi	7
3.2.1	Numeri	8
3.2.2	Caratteri	9
3.2.3	Valori diversi	9
3.2.4	Altri tipi	10
3.2.5	Tipi contenitori	10
3.2.6	Conversioni di tipo	11
3.3	Accesso ai dati e qualche formattazione	12
3.4	Operazioni matematiche e logiche	13
3.5	Manipolazione di testo e stringhe	14
3.6	Input e output standard	15
3.7	Input e output da e su file	15
3.8	Controllo del flusso	15
3.8.1	Esecuzione condizionale	15
3.8.2	Ripetizione	16
3.9	Piccoli programmi console	17
4	Il motore grafico di Red	17
4.1	GUI	18
4.1.1	Contenitore	18
4.1.2	Layout	19
4.1.3	Face	21
4.1.4	Eventi e azioni	23
4.1.5	Esempio	25
4.2	Disegno	27
4.2.1	Linee e figure	28
4.2.2	Proprietà delle linee	32
4.2.3	Manipolazione delle figure	33
4.2.4	Colore	33
4.2.5	Disegno programmato e animazione	35
4.2.6	Animazione	36

1 Predisposizione del computer

Per usare Red non dobbiamo effettuare installazioni come si usa fare con gli altri linguaggi.

Dobbiamo semplicemente copiare sul computer alcuni file, sistamarli in un luogo comodo per poterli utilizzare e, se siamo su Linux, fare alcune cose per predisporre il computer.

Red è all'indirizzo <https://www.red-lang.org/>.

Qui, oltre a trovare notizie sul linguaggio, documentazione e tutorial, nella pagina DOWNLOAD, troviamo i file che ci servono per lavorare con il linguaggio stesso.

I file sono due:

- . uno dei due, classificato CLI Red, ha un nome che incomincia per red ed è l'interprete per lavorare su normale console attraverso il terminale,
- . l'altro, classificato GUI Red, ha un nome che incomincia per red-view ed è l'interprete per lavorare su una console diversa dal terminale e, volendo, in modalità grafica.

Per i sistemi Linux e Windows abbiamo un terzo file, classificato Red Toolchain, con un nome che incomincia, appunto, con red-toolchain ed è il compilatore e fa parte di Red/System.

Possiamo semplificare le cose ribattezzando questi file con nomi più semplici, per esempio nominando red quello per lavorare su terminale, redv quello per lavorare in modalità grafica e redc il compilatore.

Un mio suggerimento per poter lavorare come si deve con Red è di creare una directory in cui collocare i file scaricati e ribattezzati e in cui posizionarci per lavorare.

Se lavoriamo su Linux non dimentichiamo di rendere eseguibili i file ricorrendo alla scheda PROPRIETÀ ► PERMESSI del gestore di file o da terminale con `chmod 555`.

Ancora, per chi lavora su Linux a 64 bit, è necessario abilitare il sistema ad operare a 32 bit. In proposito troviamo istruzioni dettagliate per le principali distro Linux nella stessa pagina da cui scarichiamo i file.

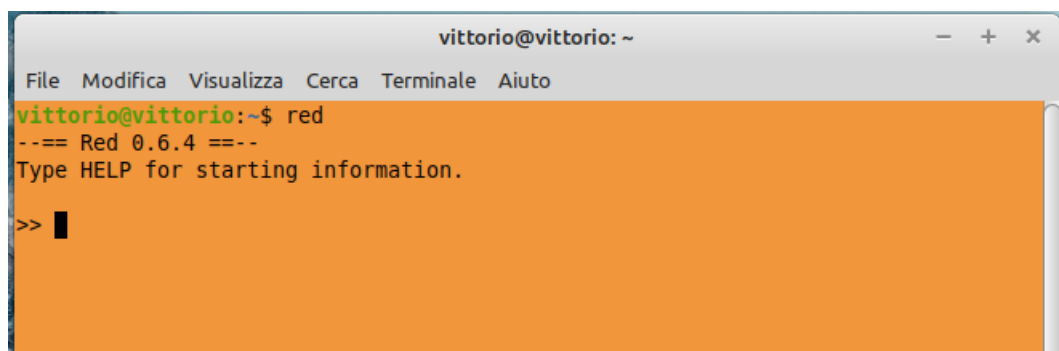
Consultando ricorrentemente la home page di Red potremo avere notizie su come evolverà tutto questo, in particolare su quando Red sarà disponibile per sistemi a 64 bit.

Un primo segnale sul fatto che siamo in presenza di qualche cosa di rivoluzionario l'abbiamo riscontrando la dimensione dei file che fanno funzionare tutto: nessuno di questi supera i 2 MB.

2 Come funziona

Abbiamo visto che disponiamo di due file eseguibili per avviare Red.

Se eseguiamo quello che ho suggerito di ribattezzare red trasformiamo il nostro terminale in una shell per il linguaggio, così



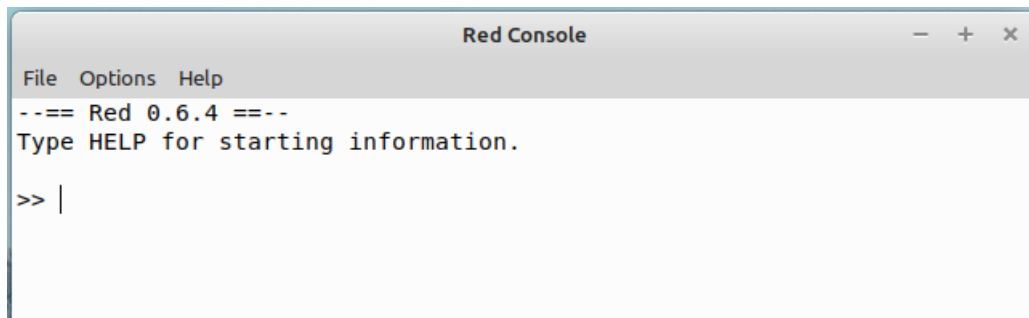
```
vittorio@vittorio: ~  
File Modifica Visualizza Cerca Terminale Aiuto  
vittorio@vittorio:~$ red  
--== Red 0.6.4 ==--  
Type HELP for starting information.  
  
>> █
```

Scrivendo al prompt istruzioni in linguaggio Red, alla pressione di INVIO ne otteniamo l'esecuzione nello stesso terminale



```
>> print "Ciao mondo"  
Ciao mondo  
>> 6 * 8  
== 48  
>> █
```

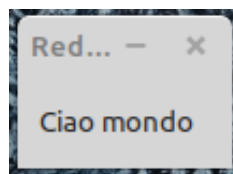
Se eseguiamo quello che ho suggerito di ribattezzare `redv` apriamo una console grafica che si presenta così



Anche in questo caso possiamo inserire al prompt istruzioni per vederle eseguite nella console alla pressione di INVIO ma, in più, possiamo lavorare con la grafica

```
>> print "Ciao mondo"
Ciao mondo
>> 6 * 8
== 48
>> view [text "Ciao mondo"]
```

Come vediamo, le prime due istruzioni vengono eseguite nella console come prima venivano eseguite nel terminale, ma l'esecuzione della terza produce questo



La shell ci consente interattività con il linguaggio, utile per imparare e sperimentare istruzioni oppure per eseguire brevi programmini, come calcoli al volo.

Veri e propri programmi vanno scritti, memorizzati in un file per eseguirli successivamente, magari compilati in codice macchina.

Per scrivere un programma in linguaggio Red ci basta un qualsiasi editor di testo (non un word processor salvo sia utilizzato per memorizzare il file in testo semplice non formattato).

Vedremo poi che cosa sia esattamente un programma in linguaggio Red e come si debba scrivere perché funzioni.

Per intanto ci basti sapere che Red è case insensitive, cioè non distingue lettere minuscole e maiuscole, per cui è indifferente, per esempio, scrivere l'istruzione `print` come `Print` o `PrInT`. Fa eccezione ciò che dobbiamo scrivere nella prima riga di un programma, dove la parola Red va scritta proprio Red non, per esempio, RED o Red.

Questa prima riga deve almeno essere

Red [] per i programmi da interpretare con l'interprete ribattezzato red (senza grafica),

Red [needs: view] per i programmi da interpretare con l'interprete ribattezzato redv.

Una volta scritto il programma lo salviamo in un file con estensione `.red` nella directory di lavoro che abbiamo dedicato a Red.

Per eseguirlo, posizionati nella directory di lavoro di Red, possiamo scrivere a terminale il nome dell'interprete (`red` o `redv`) seguito dal nome del file che contiene il programma.

Possiamo commentare una riga del programma scrivendo il commento dopo un punto e virgola (;). Per commenti più lunghi abbiamo a disposizione la funzione `comment` che possiamo usare con queste sintassi:

```
comment { .....<commento>.....}
comment [ .....<commento>.....]
comment " .....<commento>....."
```

Compilazione del programma

Come ho detto in premessa, per i sistemi Linux e Windows abbiamo a disposizione un terzo file, classificato Red Toolchain, con un nome che incomincia, appunto, con `red-toolchain`, che è il compilatore di Red/System e che ho suggerito, una volta che l'abbiamo scaricato, di rinominarlo semplicemente come `redc` e di piazzarlo nella nostra directory di lavoro dedicata a Red.

Questo file ci consente di compilare il programma scritto in linguaggio Red in modo che esso possa essere eseguito sul nostro computer con velocità molto maggiore ed anche su computer ove non è installato l'interprete Red, proprio generando un file eseguibile come possiamo fare quando utilizziamo linguaggi come C o Pascal.

Penso sia il compilatore più leggero che esista: infatti pesa solo 1,5 MB.

Vi sono tanti modi di utilizzare il compilatore. Per noi principianti ritengo basti e avanzi citarne tre, ognuno dei quali corrisponde ad una opzione da indicare in questa istruzione:

`redc <opzione> <nome_file.red>`

dove

`<nome_file.red>` è il file che contiene il programma da compilare

`<opzione>` può essere espressa con

- c per generare un eseguibile attraverso la libreria `libRedRT`,
- r per generare un eseguibile stand alone,
- t per generare un eseguibile stand alone in cross compilation.

Nel primo caso, la prima volta che usiamo il comando, viene generata la libreria `libRedRT` nella nostra directory di lavoro e la presenza di questa libreria velocizza la compilazione di successivi file.

L'eseguibile così generato può però essere utilizzato solo su un computer in cui sia presente questa libreria (cioè un computer che abbia installato l'interprete Red e sul quale sia avvenuta almeno una compilazione con l'opzione -c).

Nel secondo caso generiamo un vero e proprio eseguibile stand alone, utilizzabile anche su un computer ove non sia installato l'interprete Red.

In entrambi i casi, se siamo su Linux generiamo un eseguibile per Linux, che avrà lo stesso nome del file del programma senza l'estensione `.red`, e se siamo su Windows generiamo un eseguibile per Windows, che avrà lo stesso nome del file del programma con l'estensione `.exe`.

Nel terzo caso, su qualsiasi sistema operativo stiamo lavorando, aggiungendo all'opzione -t il nome del sistema operativo di destinazione, generiamo un eseguibile stand alone per questo sistema operativo.

Limitandoci ai tre più diffusi, i nomi dei sistemi operativi riconosciuti per questa operazione sono i seguenti

Linux per uno dei sistemi operativi GNU/Linux,

Windows per il sistema operativo Windows,

Darwin per il sistema Mac OS Intel e solo per applicazioni console,

macOS per il sistema operativo Mac OS e per applicazioni bundles.

Per esempio, se abbiamo un file di programma denominato `prova.red` e stiamo lavorando su Linux, con l'istruzione

`redc -t Windows prova.red`

generiamo l'eseguibile `prova.exe` che gira su Windows, anche senza che vi sia installato l'interprete Red.

I file compilati, fino a nuovo ordine, sono destinati a sistemi a 32 bit.

Pertanto, se siamo su un sistema Linux a 64 bit, per utilizzare l'eseguibile che abbiamo compilato, dobbiamo preventivamente installare le librerie di supporto per l'ambiente a 32 bit secondo le istruzioni che troviamo nella pagina DOWNLOAD all'indirizzo <https://www.red-lang.org/>.

3 Il linguaggio

Un programma Red è una serie di parole, che possono essere azioni (per esempio `print`) o valori (per esempio `8`).

Red vede la serie di parole come continua, ignorando i new-line che possiamo inserire con la pressione di INVIO per andare a capo quando scriviamo il programma su un editor di testo.

Red legge la serie di parole ed esegue le azioni indicate partendo da sinistra e procedendo verso destra.

Le azioni devono corrispondere ad una funzione prevista dal linguaggio e indicate usando la sintassi prevista dal linguaggio.

Possiamo conoscere la sintassi delle varie funzioni utilizzando il ricco help di Red, scrivendo nella shell di Red un punto di domanda, seguito, dopo uno spazio, dal nome della funzione.

Per esempio, se scriviamo nella shell

```
? print
```

otteniamo

```
USAGE: PRINT value
```

```
DESCRIPTION: Outputs a value followed by a newline.
```

```
PRINT is a native! value.
```

```
ARGUMENTS: value [any-type!]
```

da cui veniamo a sapere che

- . la funzione si usa richiamandone il nome seguito da un valore,
- . in output otteniamo il valore indicato seguito da a capo,
- . si tratta di una funzione nativamente prevista dal linguaggio,
- . il valore da indicare come argomento può essere di uno qualunque dei tipi riconosciuti.

I valori devono essere di un tipo riconosciuto dal linguaggio.

La loro indicazione può avvenire attraverso espressioni algebriche. Red sostituirà alle espressioni un valore. Attenzione al fatto che la valorizzazione delle espressioni viene sempre fatta procedendo da sinistra a destra senza rispettare la precedenza algebrica degli operatori. Possiamo modificare ciò con l'uso di parentesi tonde.

Esempi:

```
print "Ciao" scrive la parola Ciao ricavata dal tipo stringa "Ciao" e va a capo,
```

```
print 8 scrive il valore di tipo intero 8 e va a capo,
```

```
print 8 + 5 * 2 scrive il valore 26 e va a capo.
```

Per ottenere il corretto valore dell'espressione secondo la precedenza algebrica degli operatori, che sarebbe 18, dobbiamo indicare

```
print 8 + (5 * 2)
```

Le parole che compongono il programma vanno sempre separate da uno spazio.

Dobbiamo, per esempio, scrivere `print "Ciao"` e non `print"Ciao"`.

Così come dobbiamo scrivere `8 + 5 * 2` e non `8+5*2` o `8 + 5*2`.

E' possibile organizzare le serie di parole che compongono il programma in blocchi elencando le parole tra parentesi quadre. Questo può essere un modo per creare quelle che in altri contesti vengono chiamate routine. L'esecuzione di ciò che è racchiuso tra le parentesi può avvenire attraverso la funzione `do` oppure, in presenza di più valutazioni, `reduce`.

Se scriviamo nella shell

```
do [print "Ciao" 7 + 5 8 * 4] otteniamo Ciao 32
```

in quanto `do` esegue le istruzioni ma valuta solo l'ultima espressione contenuta nel blocco

Se scriviamo nella shell

```
reduce [print "Ciao" 7 + 5 8 * 4] otteniamo Ciao [unset 12 32]
```

3.1 Parole

La parola è la base del linguaggio Red e, abbiamo detto, può rappresentare un'azione o un valore.

In questo manualetto verremo a conoscere le principali parole chiave che identificano le funzioni proprie del linguaggio. Una di queste, che identifica la funzione `print`, l'abbiamo vista nel precedente Capitolo.

Ma possiamo noi stessi creare parole assegnando loro o un'azione o un valore.

La creazione avviene con la sintassi

`<parola>: <assegnazione>`

L'assegnazione di un'azione può avvenire attraverso un blocco di istruzioni o attraverso la creazione di una funzione.

Per esempio, per avere a disposizione una routine che mi rivolga un saluto potrei creare la parola `salutami` in questo modo

```
salutami: [print "Ciao, Vittorio, spero tu stia bene!"]
```

ed ottenere l'azione del saluto nella forma indicata ricorrendo alla funzione `do`, così

```
do salutami
```

Oppure, per avere a disposizione una funzione che raddoppi un numero, ricorrendo alla sintassi della funzione `func`, potrei creare la parola `raddoppia` in questo modo

```
raddoppia: func [n] [2 * n]
```

in modo che con l'istruzione

```
raddoppia 5
```

possa ottenere il risultato 10.

Interessante notare che nella shell dove abbiamo creato queste due parole possiamo ricorrere all'`help` per sapere come si usano.

Scrivendo

```
? salutami
```

otteniamo

```
SALUTAMI is a block!  value.  length:  2 [print "Ciao, Vittorio, spero tu sti...  
e scrivendo
```

```
? raddoppia
```

otteniamo

```
USAGE: RADDOPPIA n
```

```
DESCRIPTION: RADDOPPIA is a function!  value.
```

```
ARGUMENTS: n
```

L'assegnazione di un valore, utilizzando un tipo di dato riconosciuto dal linguaggio, può avvenire indicando un valore o una espressione per calcolarlo. In questo modo la nostra parola diventa quella che in altri linguaggi di programmazione viene chiamata variabile.

Per esempio

```
x: 5 assegna alla parola x il valore 5
```

```
y: 5 * 8 assegna alla parola y il valore 40
```

Con l'espressione

```
z: x + y assegniamo a z il valore dell'espressione (45)
```

esattamente come avviene utilizzando le variabili in altri linguaggi.

Esiste una sintassi alternativa per creare parole e assegnare ad esse un'azione o un valore:

```
set '<parola>: <assegnazione>
```

che contiene anche il modo per annullare l'assegnazione

```
unset '<parola>
```

Per esempio

```
set 'a 45 assegna alla parola a il valore 45
```

```
unset 'a annulla il valore assegnato alla parola a.
```

3.2 Tipi

Esaminiamo i principali tipi di dato riconosciuti da Red.

Il tipo viene attribuito automaticamente in relazione a ciò che scriviamo.

Esiste una funzione che ci indica il tipo attribuito:

```
type?
```

seguita da un dato o dalla parola cui è stato assegnato un dato.

Per esempio, se scriviamo al prompt della shell

type? "Ciao" otteniamo in risposta string!

Avendo una parola x con assegnato il valore 4

type? x ci dà in risposta integer!

3.2.1 Numeri

I dati numerici sono

integer!

Gli interi sono a 32 bit con segno.

Ciò significa che il valore massimo dei numeri interi con cui possiamo lavorare è compreso tra -2.147.483.648 e 2.147.483.647.

Se un numero sta fuori da questo range, Red gli attribuisce il tipo float!

float!

I numeri in virgola mobile sono a 64 bit.

Ciò significa che la precisione è tale per cui abbiamo a disposizione 16 cifre significative, virgola compresa.

Se a un intero che esce dai limiti visti prima viene attribuito il tipo float! possiamo arrivare a valutare numeri fino ad oltre 300 cifre, ma solo le prime quindici sono precise.

Vengono riconosciuti come float! i numeri scritti con una parte decimale, sia essa indicata con il separatore virgola (,) sia essa indicata con il separatore punto (.), sia essa indicata in notazione scientifica.

Scrivendo 12,6 oppure 12.6 oppure 0.126e2 oppure 0,126e2 otteniamo sempre lo stesso numero di tipo float! che, nell'usanza italiana, è 12,6.

Un interessante esercizio per sperimentare tutto ciò che riguarda i numeri può essere quello di costruirci una funzione per calcolare il fattoriale.

Non scrivo la funzione in forma ricorsiva (Red supporta benissimo la ricorsione) in quanto non mi consentirebbe di fare ciò che voglio e che poi, subito qui sotto, spiegherò ma la scrivo utilizzando un ciclo in questo modo:

```
fattoriale: func [n] [either n = 0 [1][f: 1.0 repeat i n [f: f * i]]]
```

A suo tempo vedremo i cicli e scopriremo che quello qui utilizzato fa corrispondere un ciclo a ciascun intero inferiore al numero n di cui si vuole calcolare il fattoriale, a partire dal numero più piccolo e in ciascun ciclo moltiplica il valore ottenuto prima con il numero successivo: proprio come si fa per calcolare il fattoriale.

La parola f cui viene via via assegnato il risultato del calcolo è stata inizializzata ad un valore di tipo float! (f: 1.0) in quanto, trattandosi di un valore locale, all'interno della funzione, Red non ne varierebbe il tipo in presenza di un intero oltre il suo limite fisiologico e, se la parola cui assegnare i risultati del calcolo fosse di tipo intero, ci dovremmo fermare al calcolo del fattoriale di 12, in quanto il calcolo del fattoriale di 13 genererebbe stack overflow.

Utilizzando la nostra funzione, con la forzatura verso un risultato in virgola mobile, sul computer che sto usando arrivo a calcolare il fattoriale di 170 che viene indicato con 7.257415615307994e306

un bel numero di 307 cifre, di cui sono precise solo le prime sedici, virgola compresa e già l'ultima cifra indicata (4) è sbagliata."

Sempre sul computer su cui sto lavorando, il calcolo del fattoriale di 171 utilizzando la funzione indica come risultato un infinito, senza andare in stack overflow.

3.2.2 Caratteri

I dati che rappresentano caratteri alfabetici sono

char!

Racchiuso tra doppi apici e preceduto dal simbolo # è un carattere Unicode.

In realtà si tratta del codice dei caratteri Unicode in numero intero da 0 a 1.114.111.

#"A"

è il carattere A maiuscolo e Red lo riconosce con l'intero in base decimale 65, corrispondente al codice esadecimale 0041 della tabella Unicode UTF-8.

string!

E' una serie di caratteri racchiusa tra doppi apici o tra parentesi graffe.

"Ciao bellezza" è una stringa, come {Ciao bellezza}.

3.2.3 Valori diversi

Altri tipi che identificano valori sono i seguenti

none!

Quello che in altri linguaggi è null. Indica un dato inesistente.

logic!

Ricomprende i valori true e false e gli omologhi on e yes (che valgono true) e off e no (che valgono false).

time!

Esprime un'ora che può essere indicata come ora:minuti:secondi.sottosecondi

ora: 8:45 attribuisce alla parola ora il valore 8:45:00

e, in maniera intelligente:

ora: 0:75 assegna alla parola ora il valore 1:15:00

date!

Esprime una data che può essere indicata come giorno/mese/anno o anno/mese/giorno separati con barra o lineetta.

data: 31/12/2022

data: 2022/12/31

data: 31-12-2022

data: 2022-12-31

assegnano tutti alla parola data il valore 31-Dec-2022

Red controlla che la data sia una data reale, controllando pure gli anni bisestili.

Se si scrivono i valori 29/2/2023, 31/11/2022 ecc. vengono rifiutati.

pair!

Esprime le coordinate di un punto in un sistema di assi cartesiane x y e si indica con la x e la y separate dal carattere x senza spazi di separazione.

a: 12x23 assegna alla parola a le coordinate cartesiane 12 e 23.

percent!

Serve per calcolare una percentuale e si scrive aggiungendo al numero indicante la percentuale il simbolo %.

a: 5% attribuisce alla parola a il valore 5%, cioè 0,05

a: 30 * 5% attribuisce alla parola a il valore corrispondente al 5% di 30, cioè 1,5

tuple!

E' una lista contenente da 3 a 12 elementi separati da un punto.

bianco: 255.255.255 è una tupla che identifica il colore bianco in RGB.

3.2.4 Altri tipi

url!

E' l'indirizzo di un sito web che si scrive nella forma

<protocollo>://<indirizzo>

per esempio

a: https://www.vittal.it

assegna alla parola a l'indirizzo del mio blog.

L'unico controllo che fa Red è che contenga i caratteri :// non come caratteri iniziali.

email!

E' un indirizzo email.

L'unico controllo che fa Red è che contenga un carattere @ non come primo carattere.

3.2.5 Tipi contenitori

block!

Successione di valori di qualsiasi tipo riconosciuto racchiusa tra parentesi quadrate.

Abbiamo visto nel precedente Capitolo 3 che possiamo comporre blocchi anche con istruzioni.

Red utilizza il blocco anche per quelli che in altri linguaggi si chiamano array o matrici.

a: [1 2 3 4] costruisce l'array [1 2 3 4]

m: [[3 2] [4 1] [6 2]] costruisce la matrice

$$\begin{vmatrix} 3 & 2 \\ 4 & 1 \\ 6 & 2 \end{vmatrix}$$

Gli elementi del blocco sono indicizzati a partire da 1 e si ritrovano con la sintassi qui esemplificata applicata ai blocchi a e m che abbiamo appena costruito:

a/2 ritorna 2

a/4 ritorna 4

a/5 ritorna none

m/2 ritorna [4 1]

m/1/1 ritorna 3

m/3/2 ritorna 2

vector!

L'array che abbiamo costruito prima come blocco non è direttamente assoggettabile a operazioni matematiche.

Se, utilizzando l'array `a` che abbiamo costruito nel paragrafo precedente per l'operazione

```
a * 2
```

generiamo un errore.

Per operazioni di questo tipo dobbiamo utilizzare il tipo vettore che si crea con

```
make vector! [<dato> <dato> .....]
```

dove `<dato>` può essere di tipo `integer!`, `float!`, `char!` o `percent!`

Con

```
v: make vector! [1 2 3 4]
```

creiamo il vettore `1 2 3 4` e con

```
v * 2
```

otteniamo il vettore `2 4 6 8`

map!

E' quello che in altri linguaggi si chiama dizionario ed è una sequenza di dati accoppiati ad una chiave.

Si costruisce con

```
make map! [<chiave> <dato> <chiave> <dato> .....]
```

Per ritrovare un dato si usa la funzione `select`.

Crea la map!

```
m: make map! ["Italia" "Roma" "Francia" "Parigi" "Spagna" "Madrid"]
```

con

```
select m "Francia" otteniamo "Parigi"
```

3.2.6 Conversioni di tipo

E' possibile effettuare conversioni da un tipo ad un altro utilizzando la funzione `to` con la seguente sintassi

```
to <tipo_a> <da>
```

dove

`<tipo_a>` è il tipo di destinazione della conversione,

`<da>` è il dato o la parola cui è assegnato il dato da convertire.

Esempi:

```
to integer! 3.4 ritorna 3
```

```
con x che vale 5,9 to integer! x ritorna 5
```

```
to float! 12 ritorna 12.0
```

```
to string! 14.6 ritorna "14.6"
```

```
con s che vale "13.7" to float! s ritorna 13.7
```

e per arrivare all'intero occorre fare

```
to integer! to float! s, che ritorna 13
```

Esiste anche la funzione `to-time` per convertire nel formato `time!`

```
to-time [12 78 85] ritorna 13:19:25
```

Esiste poi la funzione `as-pair` per convertire una coppia di numeri in una coppia di coordinate intere.

```
as-pair 12 25 ritorna 12x25
```

```
as-pair 3.2 5.67 ritorna 3x5
```

```
as-pair 88 12.7 ritorna 88x12
```

3.3 Accesso ai dati e qualche formattazione

Ogni parola, sia nativa del linguaggio sia creata da noi, è inserita in un dizionario.

E' il dizionario cui accede il sistema di help che abbiamo visto nel Capitolo 3.

Per vedere come il dizionario definisce una parola abbiamo anche a disposizione la funzione `get`, che si usa fornendo come argomento il nome della parola preceduto da apice semplice.

Se scriviamo nella shell

```
get 'get
```

otteniamo in risposta

```
== make native!  [[
```

```
  "Returns the value a word refers to"
```

Se scriviamo

```
get 'print
```

otteniamo la risposta

```
== make native!  [[
```

```
  "Outputs a value followed by a newline"
```

Se abbiamo creato la variabile `x` assegnandole il valore 15,5, con

```
get 'x
```

otteniamo la risposta

```
== 15.5
```

Cioè, se la parola rappresenta un'azione otteniamo la descrizione di questa azione, se la parola rappresenta un dato otteniamo il dato.

Abbiamo anche due funzioni per formattare i dati assegnati alla parola.

La funzione `mold` trasforma il dato in stringa e così lo ritorna.

Data la variabile `x` che abbiamo appena visto, con

```
mold x
```

otteniamo

```
== "15.5"
```

Data una variabile `y` con assegnato il blocco `[4 6 7]`, con

```
mold y
```

otteniamo

```
== "[4 6 7]"
```

La funzione `form` fa la stessa cosa ma depura il dato da segni come parentesi o altro.

Nel caso dell'ultimo esempio, con

```
form y
```

otteniamo

```
== "4 6 7"
```

Altro esempio di formattazione lo abbiamo con il seguente script:

```
Red []
```

```
print "-----MOLD-----"
```

```
print mold {La mia casa
```

```
è veramente bella}
```

```
print "-----FORM-----"
```

```
print form {La mia casa
```

```
è veramente bella}
```

che, eseguito, fornisce il seguente risultato

```
-----MOLD-----
```

```
"La mia casa^/è veramente bella"
```

```
-----FORM-----
```

```
La mia casa
```

```
è veramente bella
```

evidenziando come l'uso di `form` sviluppi qualche eleganza in più.

3.4 Operazioni matematiche e logiche

Ricordare sempre che Red esegue le operazioni procedendo da sinistra a destra senza rispettare alcuna precedenza degli operatori. Per precedenze diverse occorre usare le parentesi tonde.

Il separatore decimale per i numeri in virgola mobile può essere indifferentemente la virgola o il punto.

Se fa comodo, si possono inserire numeri da tastiera usando come separatore delle migliaia un apice singolo che Red ignora.

Le espressioni per eseguire le operazioni si possono rappresentare come stringa e la stringa che le contiene può essere valorizzata utilizzando la funzione `do`.

Esempi:

```
s: "7'425,5 - 2 * 8"
```

`do s` fornisce il risultato 59388.0

```
ss: "7'425,5 - (2 * 8)"
```

`do ss` fornisce il risultato 7409.5 che è quello che si otterrebbe se si rispettasse la precedenza algebrica degli operatori

Le funzioni e gli operatori che vedremo in questo Capitolo possono essere utilizzati con tutti i dati che Red rappresenta con caratteri numerici, ivi compreso il tipo `char`!

Ciò nei limiti in cui l'operazione ha senso, in caso contrario Red restituirà un errore.

Per esempio sommare o elevare a potenza valori di tipo `date`! non ha senso e Red non lo fa ma la sottrazione tra due di questi valori viene eseguita e fornisce il numero di giorni che separa due `date`.

Aritmetica di base

Per le quattro operazioni aritmetiche e l'elevamento a potenza abbiamo a disposizione operatori e funzioni.

Le funzioni si usano in notazione prefissa ed accettano solo due argomenti operandi.

Gli operatori si inseriscono tra gli operandi anche componendo lunghe espressioni.

funzione	operatore	
<code>add</code>	<code>+</code>	addizione
<code>subtract</code>	<code>-</code>	sottrazione
<code>multiply</code>	<code>*</code>	moltiplicazione
<code>divide</code>	<code>/</code>	divisione
<code>remainder</code>	<code>//</code>	resto della divisione
<code>power</code>	<code>**</code>	elevamento a potenza

Matematica

Gli argomenti passati a queste funzioni possono essere numeri, espressioni o parole con assegnato un valore.

`absolute` ritorna il valore assoluto

`exp` ritorna *e* elevato alla potenza indicata come argomento

`log-10` ritorna il logaritmo decimale

`log-2` ritorna il logaritmo in base 2

`log-e` ritorna il logaritmo naturale

`negate` ritorna il valore con segno invertito

`random` ritorna un numero casuale compreso tra 1 e il valore indicato come argomento se questo è un `integer`! oppure tra 0 e il valore indicato se questo è un `float`!

`round` ritorna l'intero più vicino

con l'opzione `/to` possiamo indicare fino a quale cifra decimale arrotondare

`round/to pi 0.001` arrotonda la costante di sistema π alla terza cifra decimale (3.142)

`square-root` estrae la radice quadrata

Trigonometria

Le funzioni dirette `sin`, `cos`, `tan` accettano l'argomento in radianti.

Le funzioni dirette `sine`, `cosine`, `tangent` accettano l'argomento in gradi.

Le funzioni inverse `asin`, `acos`, `atan` accettano l'argomento in float! e ritornano l'angolo in radianti.

Le funzioni inverse `arcsine`, `arccosine` e `arctangent` ritornano l'angolo in gradi.

Logica

Anche in questo caso abbiamo la possibilità di usare funzioni prefisse o operatori infissi.

funzione	operatore	
<code>and~</code>	<code>and</code>	e logico
<code>or~</code>	<code>or</code>	o logico

Confronto

Sempre con funzioni prefisse o operatori infissi.

funzione	operatore	
<code>equal?</code>	<code>=</code>	uguale
<code>not-equal?</code>	<code><></code>	non uguale, diverso
<code>greater?</code>	<code>></code>	maggiore
<code>greater-or-equal?</code>	<code>>=</code>	maggiore o uguale
<code>lesser?</code>	<code><</code>	minore
<code>lesser-or-equal?</code>	<code><=</code>	minore o uguale

Abbiamo poi

. la funzione `same?` (corrispondente all'operatore `=?`) che segnala verità quando i due elementi confrontati si riferiscono allo stesso dato, cioè occupano lo stesso spazio di memoria.

Per esempio se abbiamo la parola `x`: `"Ciao"` e la parola `y`: `x`, dal momento che `y` punta allo stesso dato assegnato a `x` avremo che

`same? x y` (oppure `x =? y`) ritorneranno `true`.

. la funzione `strict-equal?` (corrispondente all'operatore `==`) che segnala verità quando vi è perfetta corrispondenza tra i due elementi confrontati. Per esempio, in un contesto dove maiuscole e minuscole non si distinguono, come avviene con Red, se abbiamo le due stringhe `a`: `"Ciao"` e `b`: `"ciao"`

con `equal? a b` (oppure `a = b`) avremo `true`

con `strict-equal? a b` (oppure `a == b`) avremo `false`

3.5 Manipolazione di testo e stringhe

Qui elenco le principali funzioni per lavorare con testi e stringhe.

L'argomento da assegnare che indico con `<a>` può essere una stringa o una parola cui è assegnata una stringa.

Ricordo che l'indice dei caratteri che formano la stringa parte da sinistra con il numero 1.

`length? <a>` ritorna un intero che indica il numero di caratteri della stringa,

`insert <a> <stringa>` aggiunge una stringa all'inizio di un'altra,

`append <a> <stringa>` aggiunge una stringa alla fine di un'altra,

`insert at <a> <indice> <stringa>` inserisce una stringa in un'altra nel punto indicato,

`rejoin [<a> <a> <a> ...]` concatena stringhe,

`trim <a>` rimuove spazi eventualmente presenti all'inizio e alla fine di una stringa,

`trim/head <a>` rimuove spazi eventualmente presenti all'inizio di una stringa,

`trim/tail <a>` rimuove spazi eventualmente presenti alla fine di una stringa,

`trim/lines <a>` rimuove indicazioni di end of line da una stringa,

`trim/with <a> "<carattere/i>"` rimuove da una stringa il carattere o i caratteri indicati.

3.6 Input e output standard

L'input e l'output standard sono rispettivamente quelli dalla tastiera e verso lo schermo.

Per l'input abbiamo a disposizione due funzioni:

`input` legge quanto scritto su tastiera, dopo premuto INVIO, come stringa,

`ask <stringa>` lo stesso, con possibilità di scrivere un messaggio per l'input.

Se si vuole registrare l'input in tipo diverso da stringa! occorre anteporre alla chiamata della funzione una istruzione per la conversione voluta.

Esempi:

`nome: input` assegna a `nome` il valore scritto su tastiera come stringa,

`nome: ask "Come ti chiami? "` lo stesso, premettendo la richiesta del nome,

`x: to float! input` assegna a `x` il valore scritto su tastiera come numero in virgola mobile,

`y: to integer! ask "Scegli un numero intero "` assegna a `y` il numero intero digitato.

Per l'output le principali funzioni sono le seguenti.

`print` riceve come argomento ciò che vogliamo scrivere sullo schermo, lo scrive a va a capo.

L'argomento può essere una stringa o un'espressione, che viene valutata prima di scriverla;

oppure può essere una parola e verrà scritto il valore ad essa assegnato.

`prin` lo stesso di `print` con la differenza che scrive e non va a capo,

`probe` scrive l'argomento senza effettuare alcuna valutazione.

Esempi:

`print "Ciao"` scrive la parola Ciao e va a capo,

`print 3 * 4` scrive 12 e va a capo,

`prin [4 + 5]` scrive 9 e non va a capo,

`probe [4 + 5]` scrive [4 + 5]

3.7 Input e output da e su file

Per lavorare con un file occorre sempre indicare il percorso per arrivare al file stesso antepo-
nendo il simbolo % alla serie di directory separate dalla barra /.

Per arrivare al file prova che si trova nella mia directory home nella sottodirectory Documenti
dovrò indicare

`%/home/vittorio/Documenti/prova`

Leggiamo il contenuto di un file con la funzione

`read <percorso>`

Se assegniamo quanto letto a una parola, stampando il valore di quella parola vediamo l'intero
contenuto del file così come è.

Scriviamo in un file con la funzione

`write <percorso> <stringa_di_testo>` Se il file non c'è viene creato. Se c'è viene sovrascritto.

`write/append <percorso> <stringa_di_testo>` Aggiunge contenuto a un file esistente.

3.8 Controllo del flusso

3.8.1 Esecuzione condizionale

`if <test> <blocco_istruzioni>`

se <test> è vero viene eseguito il blocco di istruzioni, altrimenti si passa oltre,

`either <test> <blocco_istruzioni_per_vero> <blocco_istruzioni_per_falso>`

se <test> è vero viene eseguito il primo blocco, altrimenti viene eseguito il secondo,

`switch <valore> <blocco_alternative>`

esegue il blocco di istruzioni corrispondente a un certo valore,

`switch/default <valore> <blocco_alternative> <blocco_default>`

esegue il blocco di default se nessuna alternativa è verificata,

`case [<test> <blocco> <test> <blocco> <test> <blocco> . . . .]`

fa una serie di test ed esegue il blocco corrispondente alla prima verità,

`case/all [<test> <blocco> <test> <blocco> <test> <blocco> . . . .]`

fa una serie di test ed esegue tutti i blocchi corrispondenti a verità.

Esempi:

```
>> if 4 < 5 [print "OK"]
OK
>> either 4 > 5 [print "OK"] [print "NO"]
NO
>> switch 20 [10 [print "dieci"] 20 [print "venti"]]
venti
>> switch/default 30 [10 [print "dieci"] 20 [print "venti"]] [print "non so"]
non so
>> case [1 > 2 [print "male"] 2 > 1 [print "bene"] 3 > 1 [print "benissimo"]]
bene
>> case/all [1 > 2 [print "male"] 2 > 1 [print "bene"] 3 > 1 [print "benissimo"]]
bene
benissimo
```

3.8.2 Ripetizione

`loop n <blocco_istruzioni>`
esegue le istruzioni contenute nel blocco n volte,
`repeat i n <blocco_istruzioni>`
lo stesso di `loop` gestendo un indice che si incrementa ad ogni giro,
`foreach i a <blocco_istruzioni>`
data una parola a con assegnato un blocco esegue istruzioni per ogni elemento del blocco,
`while [<condizione>] <blocco_istruzioni>`
esegue le istruzioni fino a quando una condizione è vera,
`until [<blocco_istruzioni> <condizione>]`
esegue le istruzioni fino a quando il blocco restituisce verità.

Esempi:

```
>> loop 2 [print "Ciao"]
Ciao
Ciao
>> repeat i 2 [prin i print " Ciao"]
1 Ciao
2 Ciao
>> a: [1 2 3]
>> foreach i a [print i * i]
1
4
9
>> i: 1
>> while [i < 3] [print "Ciao" i: i + 1]
Ciao
Ciao
>> i: 2
>> until [print "Ciao" i: i - 1 i = 0]
Ciao
Ciao
== true
```


3.9 Piccoli programmi console

Ciò che abbiamo visto sin qui ci mette in condizione di scrivere programmi in linguaggio Red per console, cioè eseguibili su terminale.

Questo programma chiede il raggio di un cerchio per calcolare circonferenza e area del cerchio ed espone i risultati con tre cifre decimali:

```
Red []
r: to float! ask "Indica il raggio di un cerchio: "
c: 2 * pi * r
a: r * r * pi
prin "Per un cerchio di raggio " print r
prin "la circonferenza è " print round/to c 0.001
prin "l'area è " print round/to a 0.001
```

Quest'altro traduce in inglese i giorni della settimana:

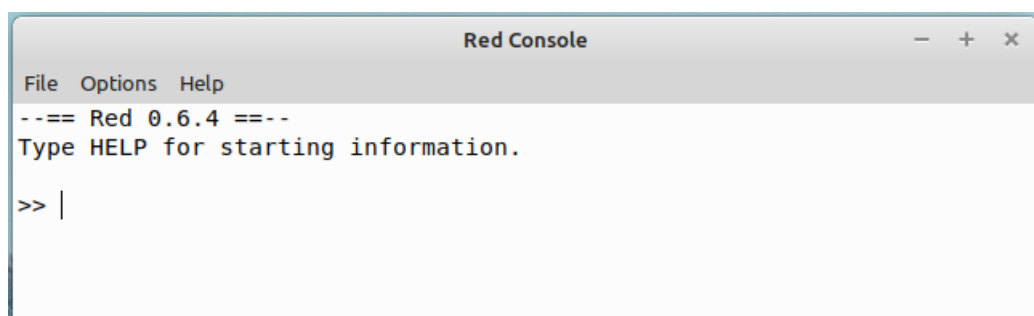
```
Red []
giorno: ask "Scrivi un giorno della settimana in italiano: "
switch/default giorno [
  "lunedì" [print "in inglese si dice Monday"]
  "martedì" [print "in inglese si dice Tuesday"]
  "mercoledì" [print "in inglese si dice Wednesday"]
  "giovedì" [print "in inglese si dice Thursday"]
  "venerdì" [print "in inglese si dice Friday"]
  "sabato" [print "in inglese si dice Saturday"]
  "domenica" [print "in inglese si dice Sunday"]
]
[print "Non ho capito"]
```

4 Il motore grafico di Red

Il sistema grafico per il linguaggio di programmazione Red si chiama View Graphic Engine, o semplicemente View.

Esso è disponibile utilizzando il file che ho consigliato di ribattezzare brevemente come redv.

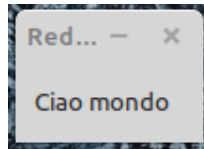
Abbiamo già visto che se scriviamo a terminale il nome di questo file apriamo una console grafica, che assomiglia al terminale di sistema ma è invece un oggetto grafico di Red/View



nella quale possiamo scrivere istruzioni in linguaggio Red e vederle eseguite nella console stessa, come fossimo sul terminale di sistema aperto su Red, ma possiamo anche scrivere istruzioni nel dialetto chiamato VID con cui realizziamo oggetti grafici.

```
>> print "Ciao mondo"
Ciao mondo
>> 6 * 8
== 48
>> view [text "Ciao mondo"]
```

Nell'illustrazione, le prime due istruzioni sono in linguaggio Red e sono eseguite in console, la terza è in linguaggio VID, incorporato nel linguaggio Red, e produce questo



Se scriviamo il programma con un editor di testo per eseguirlo poi, compilato o meno, dobbiamo richiamare il motore grafico nella prima riga del file di programma con

```
Red [needs: view]
```

Il resto del programma sarà un misto di linguaggio Red e di linguaggio VID se stiamo lavorando ad un programma con GUI o sarà quasi interamente in linguaggio Draw, il linguaggio nel linguaggio per disegnare, se stiamo lavorando ad un programma per eseguire semplici disegni.

La grafica in Red ha un suo gergo particolare nel quale gli elementi con cui si costruiscono gli oggetti grafici, che penso universalmente si chiamino widgets, qui si chiamano faces.

Le caratteristiche di questi oggetti, che in altri contesti esprimiamo attraverso l'indicazione di opzioni, in Red si indicano facendo seguire al nome del face una serie di dati, chiamati facet, dal cui tipo Red capisce che caratteristica indicano e la attribuisce al face: per esempio, se il dato è di tipo pair! ci si riferisce alla dimensione del face, se il dato è di tipo string! ci si riferisce a un testo da scrivere sul face.

Ancora a seguire, se al face indicato vogliamo abbinare l'esecuzione di qualche azione, indichiamo l'azione stessa in un blocco.

Per esempio, con

```
button 50x20 "ESCI" [quit]
```

creiamo un pulsante di dimensione 50 x 20 in pixel con sopra scritto ESCI, premendo il quale chiudiamo la finestra e abbandoniamo il programma.

Come si vede, anche nella grafica Red manifesta la propria assoluta originalità.

Si vede anche che l'istruzione, che è scritta in linguaggio VID, per essere compresa appieno richiede la conoscenza delle basi del linguaggio Red. Solo così sappiamo che 50x20 esprime un valore di tipo pair!, che una parola scritta tra doppi apici è di tipo string! e che un blocco è qualche cosa di inserito tra parentesi quadre.

4.1 GUI

Un programma corredato da interfaccia utente grafica tipicamente si compone, nell'ordine, dei seguenti elementi:

- . creazione e settaggio del contenitore,
- . definizione del layout,
- . inserimento dei face con eventuale indicazione delle azioni collegate.

4.1.1 Contenitore

Il comando costruttore della finestra contenitore è

```
view []
```

Con questo comando generiamo una finestrella senza titolo, a sfondo grigio chiarissimo, al centro dello schermo, con uno spazio contenitore di 80x80 pixel che automaticamente si allarga per contenere tutti i face che vi inseriamo.

Per avere una finestra contenitore come la vogliamo noi abbiamo a disposizione una serie di strumenti.

Innanzitutto, all'interno delle parentesi quadre che seguono il comando costruttore possiamo indicare queste caratteristiche della finestra che vogliamo:

`title` seguito da una stringa indicante il titolo della finestra,

`size` seguito dal pair! con cui indicare le dimensioni in pixel della finestra,

backdrop seguito dal nome inglese di un colore¹ o da una tupla RGB per indicare il colore di fondo della finestra.

Con

```
view [title "FINESTRA DI PROVA" size 300x200 backdrop green]
```

otteniamo la finestra



collocata al centro dello schermo

(avremmo ottenuto lo stesso colore con backdrop 0.255.0).

Se vogliamo collocare la finestra in posizione diversa dal centro dello schermo dobbiamo introdurre l'opzione offset con questa sintassi:

```
view/options [<caratteristiche>] [offset: <posizione>]
```

dove <posizione> è un pair! che indica la posizione sullo schermo dell'angolo in alto a sinistra della finestra.

Con

```
view/options [title "FINESTRA DI PROVA" size 300x200 backdrop green] [offset: 30x50]
```

otteniamo la finestra di prima, collocata a 30 pixel dal bordo sinistro dello schermo e a 50 pixel dal bordo superiore (l'origine delle coordinate dello schermo è infatti nell'angolo in alto a sinistra).

4.1.2 Layout

All'interno del contenitore dobbiamo sistemare i face che servono per comporre la nostra GUI.

I face si inseriscono scrivendoli uno via l'altro, eventualmente seguiti dai relativi facet.

Il layout di default, che Red applica se non indichiamo altro, si chiama across e dispone i face uno dopo l'altro nel contenitore partendo da sinistra.

¹I nomi di colori riconosciuti da Red sono i seguenti

	aqua		beige		black		blue
	brick		brown		coal		coffee
	crimson		cyan		forest		gold
	gray		green		ivory		khaki
	leaf		linen		magenta		maroon
	mint		navy		oldrab		olive
	orange		papaya		pewter		pink
	purple		reblue		rebolor		red
	violet		white		yellow		

Un semplice layout alternativo si chiama `below` e dispone i face uno sotto l'altro partendo dall'alto.

Nell'elencazione dei face possiamo inserire la parola `return` quando vogliamo che la collocazione dei face passi alla riga successiva se siamo in modalità `across` o alla colonna successiva se siamo in modalità `below`.

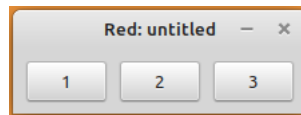
Esempi:

```
view [across button "1" button "2" button "3"]
```

o, semplicemente,

```
view [button "1" button "2" button "3"]
```

dispone i tre pulsanti così



```
view [below button "1" button "2" button "3"]
```

dispone i tre pulsanti così

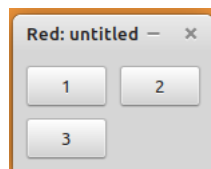


```
view [across button "1" button "2" return button "3"]
```

o, semplicemente

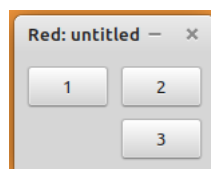
```
view [button "1" button "2" return button "3"]
```

dispone i tre pulsanti così



```
view [below button "1" return button "2" button "3"]
```

dispone i tre pulsanti così



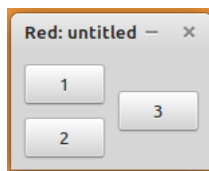
Si tratta di modelli di layout così detti relativi, nei quali il posizionamento di un face avviene in conseguenza del posizionamento del face che lo precede.

Molto spesso, per costruire GUI sofisticate, conviene ricorrere al modello di layout con posizionamento assoluto, nel quale scegliamo noi dove va posizionato il face facendo precedere al suo nome la parola chiave `at` seguita da un `pair!` che indica il punto dove posizionare l'angolo in alto a sinistra del face.

Esempio:

```
view [at 10x10 button "1" at 10x50 button "2" at 80x30 button "3"]
```

dispone i pulsanti così



Per GUI molto sofisticate e per alleggerire il lavoro per determinare e indicare i punti di ancoraggio dei face può convenire suddividere il contenitore in più zone utilizzando il `face panel`, che vedremo subito, applicando ad ogni zona un modello di layout diverso, scegliendo il più conveniente e meno laborioso per ciò che vogliamo mettere nella zona stessa.

4.1.3 Face

Per utilizzare i face occorre sapere a cosa servono e quali sono i facet ai quali ogni face è sensibile: non tutti i facet, infatti, si adattano a ciascun face. Per esempio il `face button` non può essere colorato ma dobbiamo prenderlo con il colore grigio chiaro come ce lo passa il convento.

Una cosa molto importante da ricordare: il face è un oggetto e può essere assegnato ad una parola.

In simile contesto, i facet sono suoi attributi e a ciascuno di essi si accede con una notazione composta dal nome della parola cui è assegnato il face e dal nome del facet separati da una barra (/).

Per esempio, se è stato assegnato il `face button "OK"` alla parola `b` con

```
b: button "OK"
```

`b/text` ritorna OK

I face che ci offre Red sono moltissimi.

Cominciamo da quello richiamato alla fine del precedente Capitolo e che ci consente di suddividere il contenitore in più zone:

panel E' un face contenitore di altri face.

Gli si può dare una dimensione con un `pair!` e un colore di sfondo con un nome di colore o una tupla RGB e si dispongono i face al suo interno elencandoli in un blocco.

Con

```
view [below panel 150x50 yellow [button "1" button "2"] panel 150x100 255.0.0 [below button "3" button "4"]]
```

otteniamo



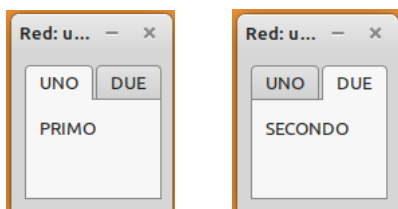
Possiamo inserire i `panel` in pagine diverse ricorrendo al `face`

tab-panel I `panel` non si dispongono sulla stessa pagina del contenitore ma in pagine diverse e sovrapposte, apribili cliccando sulla linguetta che le contraddistingue.

Con

```
view [tab-panel 100x100 ["UNO" [text "PRIMO"] "DUE" [text "SECONDO"]]]
```

otteniamo



dove la figura di destra si ottiene cliccando sulla linguetta DUE della figura di sinistra.

Ora vediamo i tre face che ricorrono in ogni GUI: il testo come etichetta, la finestra per l'immissione di testo e il pulsante.

text E' quello che in altri linguaggi si chiama label.

Nelle GUI serve per inserire scritte destinate a istruzioni o a esposizione di risultati di elaborazioni.

Il testo si indica con una stringa e si può dare una dimensione allo sfondo con un pair! e un colore di sfondo con un nome di colore o una tupla RGB.

Per le caratteristiche del font abbiamo invece a disposizione tre istruzioni che vanno indicate dopo i facet:

- . font-name seguita da un nome di font installato sul sistema su cui lavoriamo,
- . font-size seguita da un numero intero indicante la dimensione del font in punti,
- . font-color seguita da nome di un colore o da una tupla RGB ad indicare il colore del font.

Con

```
view [text "Ciao" cyan 90x30 font-name "eufm10" font-size 35 font-color red]  
ottendiamo
```

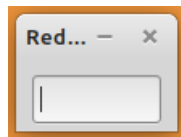


field Serve per immettere dati in forma di testo da tastiera su una sola riga.

La relativa finestrella può essere dimensionata con un pair! ed ha sfondo bianco immutabile.

Con

```
view [field 80x25]  
ottendiamo
```

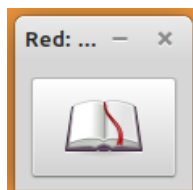


button E' il classico pulsante su cui si clicca per ottenere qualcosa.

Può essere dimensionato con un pair! e si può fare in modo che sia contraddistinto da una scritta, indicabile con una stringa, o da una icona/immagine, indicandone percorso e nome preceduti dal carattere %.

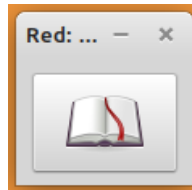
Con

```
view [button 90x30 "ESCI"]  
ottendiamo
```



Con

```
view [button 90x60 %/home/vittorio/Immagini/dictionary.png]  
ottendiamo
```



Abbiamo poi due utilissimi face che possono servire per applicazioni d'ufficio o grafiche.

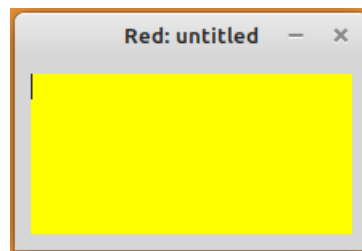
area Serve per immettere testo su più righe.

Può essere dimensionata con un pair! ed ha per default sfondo bianco, che può essere modificato con un nome o con una tupla RGB.

Con

```
view [area 200x100 yellow]
```

otteniamo



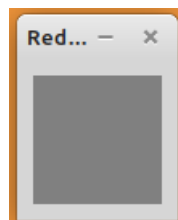
base Serve per creare altri face e per disegnare.

E' alla base della grafica di disegno di Red che vedremo più avanti.

Per default, con

```
view [base]
```

si presenta così



con una dimensione di 80x80 pixel e sfondo grigio.

Aggiungendo un pair! possiamo scegliere una diversa dimensione e possiamo scegliere un colore diverso indicandone il nome o attraverso una tupla RGB.

Altri face Per altri face di arricchimento delle GUI, come check, radio, text-list, drop-down, drop-list, progress, slider, rimando alla documentazione ufficiale.

4.1.4 Eventi e azioni

Quando parliamo di eventi che hanno a che fare con una GUI alludiamo a manipolazioni effettuate attraverso la tastiera (pressione di uno o di determinati tasti) o attraverso il mouse (puntamento e/o pressione di uno dei tasti o uso della rotella del mouse).

Dalla documentazione ufficiale di Red possiamo vedere che gli eventi gestiti da questo linguaggio sono tantissimi. In questo manualetto mi limito a presentare quelli più comunemente usati:

- . pressione del tasto sinistro del mouse (in Red denominato `down` o `click`),
- . pressione del tasto INVIO della tastiera (in Red denominato `enter`),
- . pressione di un qualsiasi tasto della tastiera (in Red denominato `change` o `key`).

Ciascuno dei face che abbiamo visto nel precedente paragrafo è predisposto per gestire un evento di default:

- . pressione del tasto sinistro del mouse per il face `text`,
- . pressione del tasto INVIO della tastiera per il face `field`,
- . pressione del tasto sinistro del mouse per il face `button`,
- . pressione di un qualsiasi tasto della tastiera per il face `area`,
- . pressione del tasto sinistro del mouse per il face `base`.

Per abbinare azioni a questi eventi basta far seguire al nome del face e degli eventuali facet un blocco, che in Red si chiama `action facet`, in cui, tra parentesi quadre, si indica il codice da eseguire.

Con

```
view [button "PROVA" [print "Ciao"]]
```

apriamo una finestra in cui compare un pulsante con la scritta PROVA e cliccando su di esso scriviamo la parola Ciao nella console di Red.

Se vogliamo che il pulsante reagisca ad un evento diverso da quello di default, appena prima del blocco per l'azione dobbiamo inserire la parola `on` seguita, separando con trattino, dal nome dell'evento voluto.

Per esempio, tra i tanti, Red prevede l'evento `over` che consiste nel passaggio del mouse sopra il face.

Con

```
view [button "PROVA" on-over [print "Ciao"]]
```

apriamo una finestra in cui compare un pulsante con la scritta PROVA e semplicemente passando il mouse su di esso scriviamo la parola Ciao nella console di Red.

Negli esempi l'azione richiesta coinvolge la console e questo serve a dimostrare che si può fare, anche se contraddice l'uso della GUI, e soprattutto alla semplificazione degli esempi stessi.

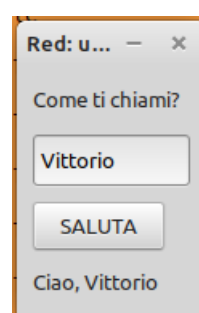
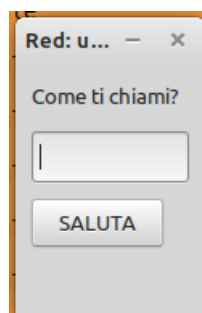
Per avere anche il risultato delle elaborazioni nella GUI si deve lavorare un po' di più sfruttando il fatto che, come ho richiamato all'inizio del paragrafo precedente, i face sono oggetti e i facet sono loro attributi.

Questo è un programmino, semplificato nell'aspetto grafico per rimanere all'essenziale, in cui viene chiesto il nome all'utente per salutarlo

```
Red [needs: view]
```

```
view [  
  below  
  text "Come ti chiami?"  
  i: field  
  button "SALUTA" [  
    nome: i/text  
    insert o/text rejoin ["Ciao, " nome]  
  ]  
  o: text ""  
]
```

La figura di sinistra mostra la GUI quando il programma parte e la figura di destra mostra la GUI dopo che l'utente ha inserito il nome nell'apposita finestra e premuto il pulsante SALUTA:



Con la sintassi minimale tipica del linguaggio Red, abbiamo la scelta del layout (below), la creazione di un'etichetta che invita ad inserire il nome, la creazione di un campo di inserimento testo assegnato alla variabile `i` (come input), la creazione di un pulsante con scritto SALUTA e la creazione con inizializzazione a stringa vuota di un'etichetta assegnata alla variabile `o` (come output).

Il blocco eseguibile in corrispondenza all'evento di default «pressione del tasto sinistro del mouse» sul pulsante crea una variabile nome assegnandole il testo inserito nel campo di inserimento `i` e finalmente aggiunge alla stringa vuota che costituisce il contenuto dell'etichetta `o` la concatenazione tra la stringa "Ciao, " e la stringa assegnata alla parola `nome`: più difficile descriverlo che farlo.

Ovviamente i nomi delle variabili possono essere scelti con qualsiasi criterio e l'indentazione che si vede nel modo con cui ho riprodotto il programma è stata adottata unicamente per rendere più chiaro il contenuto ad occhio umano.

Rammento che Red legge il programma prescindendo dagli a capo, come se fosse scritto tutto su una sola lunga riga: importante che vi sia sempre uno spazio che divide le parole.

4.1.5 Esempio

Per riepilogare un po' tutto propongo un programma non banalissimo che ci può servire, soprattutto per la scelta dei colori di tutto ciò che faremo nel prossimo Capitolo.

Il programma è questo:

```
Red [needs: view]
view
[title "COLORI" tab-panel 200x200
  ["COLORI RED" [
    b1: base 70x150 white
    t: text-list 100x150 data ["aqua" "beige" "black" "blue" "brick" "brown" "coal"
      "coffee" "crimson" "cyan" "forest" "gold" "gray" "green" "ivory" "khaki" "leaf"
      "linen" "magenta" "maroon" "mint" "navy" "oldrab" "olive" "orange" "papaya"
      "pewter" "pink" "purple" "reblue" "rebolor" "red" "violet" "white" "yellow" " "]
    [switch t/selected [
      1 [b1/color: aqua]
      2 [b1/color: beige]
      3 [b1/color: black]
      4 [b1/color: blue]
      5 [b1/color: brick]
      6 [b1/color: brown]
      7 [b1/color: coal]
      8 [b1/color: coffee]
      9 [b1/color: crimson]
      10 [b1/color: cyan]
      11 [b1/color: forest]
      12 [b1/color: gold]
      13 [b1/color: gray]
      14 [b1/color: green]
      15 [b1/color: ivory]
      16 [b1/color: khaki]
      17 [b1/color: leaf]
      18 [b1/color: linen]
      19 [b1/color: magenta]
      20 [b1/color: maroon]
      21 [b1/color: mint]
      22 [b1/color: navy]
      23 [b1/color: oldrab]
      24 [b1/color: olive]
      25 [b1/color: orange]
      26 [b1/color: papaya]
      27 [b1/color: pewter]
      28 [b1/color: pink]
      29 [b1/color: purple]
      30 [b1/color: reblue]
```

```

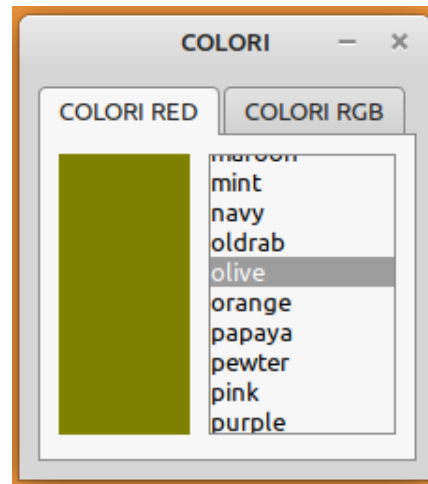
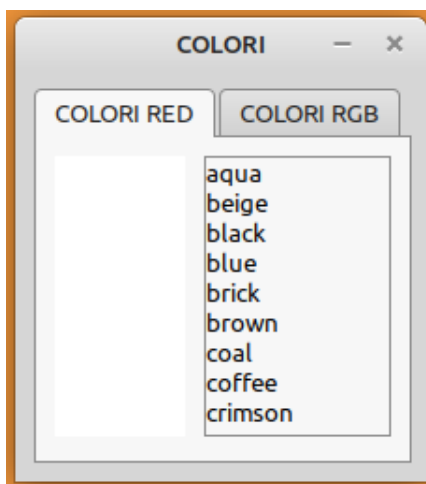
31 [b1/color: rebolor]
32 [b1/color: red]
33 [b1/color: violet]
34 [b1/color: white]
35 [b1/color: yellow]
]
]
]
"COLORI RGB" [
  text 60x15 " Rosso" text 60x15 " Verde" text 60x15 " Blu" return
  i1: field 55x15 i2: field 55x15 i3: field 55x15
  at 59x75 button "COLORE" [
    x1: to integer! i1/text
    x2: to integer! i2/text
    x3: to integer! i3/text
    x: 1.1.1
    x/1: x1
    x/2: x2
    x/3: x3
    b2/color: x
  ]
  at 10x115 b2: base 184x52 white
]
]
]

```

Esso serve per verificare visivamente il colore corrispondente ad una delle parole identificative del linguaggio Red oppure corrispondente ad una qualsiasi combinazione di valori nel sistema RGB, dove ogni colore corrisponde al mix di certe quantità (da 0 a 255) dei colori base Rosso Verde e Blu.

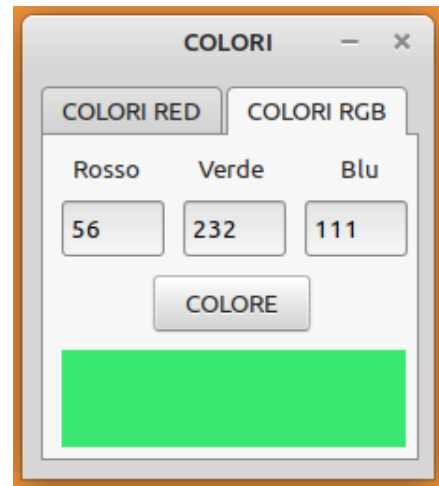
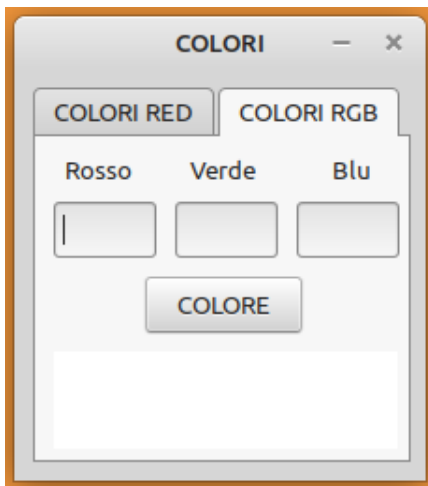
Al lancio del programma ci si presenta la GUI di sinistra, aperta sulla finestra dedicata a verificare il colore corrispondente alle 35 parole identificative del linguaggio Red.

Sulla destra abbiamo la stessa finestra dopo che abbiamo scelto nell'elenco la parola olive



Cliccando sulla linguetta colori rgb apriamo la finestra che compare sotto a sinistra, dedicata a verificare il colore corrispondente ad una combinazione RGB.

Sulla destra abbiamo la stessa finestra dopo che abbiamo inserito la combinazione 56 per il Rosso, 232 per il Verde e 111 per il Blu



Ognuna delle tre finestrelle per comporre la combinazione deve contenere un numero intero tra 0 e 255 prima di premere il pulsante COLORE per vedere il risultato della combinazione.

Lasciando al lettore l'esercizio di capire i vari passi del programma, richiamo soltanto l'attenzione sull'utilizzo del `face text-list`, che ho semplicemente nominato senza altri commenti nel paragrafo 13.3.8 tra gli Altri face, e sul meccanismo con cui è costruita la tupla per generare il colore in combinazione RGB.

Per avere un oggetto di tipo tupla è stato necessario costruirne una con valori provvisori e la giusta sintassi (`x: 1.1.1`) e poi sostituire i valori provvisori con quelli letti nelle finestrelle di immissione dati.

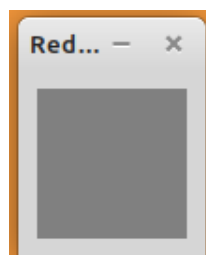
Per il resto si tratta di cose contenute, per la parte grafica, in questo manuale e, per la parte di trattamento dei dati, nel manuale `red.pdf` reperibile sul mio blog all'indirizzo www.vittal.it.

4.2 Disegno

Per disegnare occorre un piano su cui farlo e Red, a questo scopo, ci mette a disposizione il `face base` che abbiamo visto nel precedente Capitolo.

Come abbiamo appreso nel paragrafo 13.3.7 esso si costruisce con un blocco `view`

`[base]`
e, in mancanza di altre indicazioni, si presenta così



con una dimensione di 80x80 pixel e sfondo grigio, che ricorda il colore della lavagna.

Aggiungendo un `pair!` possiamo scegliere una diversa dimensione e possiamo scegliere un colore diverso indicandone il nome o attraverso una tupla RGB.

L'origine dei pixel si trova nell'angolo in alto a sinistra, le cui coordinate sono 0 e 0, che si indicano in Red con il `pair! 0x0`. La prima cifra si riferisce all'asse orizzontale e la seconda cifra si riferisce all'asse verticale.

Con `10x20` indichiamo il pixel (praticamente un punto) a 10 pixel da sinistra e a 20 pixel dall'alto della lavagna.

4.2.1 Linee e figure

Per disegnare linee e figure facciamo seguire al face base il facet draw e un blocco che contiene le istruzioni per come costruire la linea o la figura, secondo la sintassi che indico nel seguito per ciascun tipo di linea o di figura.

Importante ricordare che il face è un oggetto e il facet è un metodo di quell'oggetto. Per cui se assegniamo alla parola a istruzioni per un disegno con

a: base draw [<istruzioni>]

con a/draw accediamo al blocco [<istruzioni>]

Questo è molto utile per animare ciò che disegniamo.

line Si costruisce con

line <punto> <punto>

dove i due punti, indicati con due pair!, sono i punti di partenza e di arrivo della linea.

Indicando più punti possiamo costruire una linea spezzata.

Con

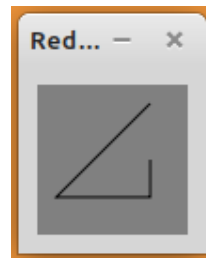
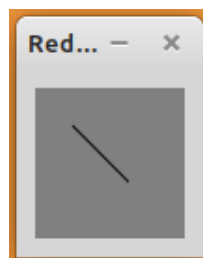
```
view [base draw [line 20x20 50x50]]
```

otteniamo la figura sulla sinistra.

Con

```
view [base draw [line 60x10 10x60 60x60 60x40]]
```

otteniamo la figura sulla destra.



triangle Si costruisce con

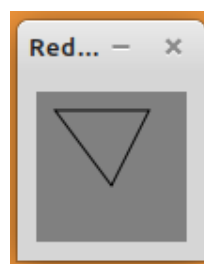
triangle <punto> <punto> <punto>

dove i tre punti, indicati con altrettanti pair!, sono i vertici del triangolo.

Con

```
view [base draw [triangle 10x10 40x50 60x10]]
```

otteniamo



box Rettangoli e quadrati e si costruiscono con

box <punto> <punto>

dove i due punti, indicati con due pair!, corrispondono rispettivamente allo spigolo in alto a sinistra e allo spigolo in basso a destra della figura.

Possiamo aggiungere un numero intero per indicare il raggio di arrotondamento degli spigoli.

Con

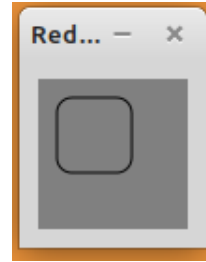
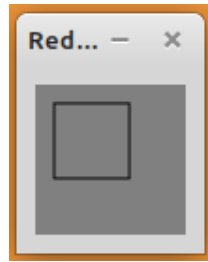
```
view [base draw [box 10x10 50x50]]
```

otteniamo la figura sulla sinistra.

Con

```
view [base draw [box 10x10 50x50 8]]
```

otteniamo la figura sulla destra



polygon Si costruisce con

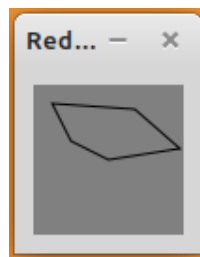
```
polygon <punto> <punto> <punto> ....
```

dove i punti, indicati pair!, sono i vertici del poligono e la chiusura avviene automaticamente.

Con

```
view [base draw [polygon 10x10 20x30 40x40 78x34 54x13]]
```

otteniamo



circle Si costruisce con

```
circle <punto> <raggio>
```

dove <punto> è il pair! indicante il centro del cerchio e <raggio> è un numero che indica il raggio del cerchio stesso.

Indicando il raggio con il numero 1 costruiamo un punto localizzato dove indicato dal pair!

Se indichiamo due raggi costruiamo una ellisse avente come asse orizzontale il primo indicato e come asse verticale il secondo.

Con:

```
view [base draw [circle 40x40 1]]
```

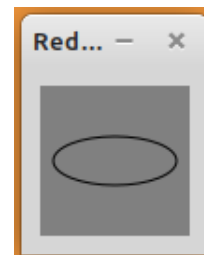
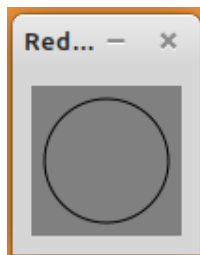
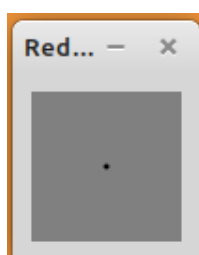
otteniamo la figura di sinistra,

```
view [base draw [circle 40x40 32,5]]
```

otteniamo la figura centrale,

```
view [base draw [circle 40x40 30 12.8]]
```

otteniamo la figura di destra.



ellipse Possiamo costruire una ellisse anche con

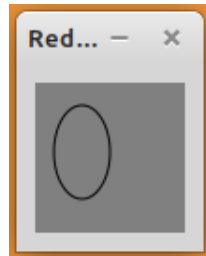
`ellipse <punto> <punto>`

dove i due punti, indicati da altrettanti pair!, sono lo spigolo in alto a sinistra e lo spigolo in basso a destra di un rettangolo immaginario in cui è inscritta l'ellisse voluta.

Con

```
view [base draw [ellipse 10x12 30x50]]
```

otteniamo



arc Costruisce l'arco su un cerchio o ellisse immaginari con

`arc <punto> <raggi> <inizio> <fine>`

dove

`<punto>` è un pair! indicante il centro del cerchio o dell'ellisse,

`<raggi>` è un pair! il cui numero di sinistra indica il raggio orizzontale e il numero di destra indica il raggio verticale (se i due numeri sono uguali avremo un cerchio, altrimenti una ellisse,

`<inizio>` è un intero indicante, in gradi, l'angolo di inizio dell'arco,

`<fine>` è un intero indicante, in gradi, l'angolo di fine dell'arco.

Gli angoli di inizio e fine dell'arco si muovono in senso orario partendo dall'asse orizzontale a destra del centro.

Aggiungendo la parola `closed` otteniamo la chiusura dell'arco con due linee con origine nel centro.

Con

```
view [base draw [arc 40x40 30x30 90 160]]
```

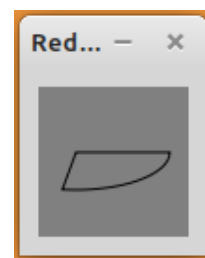
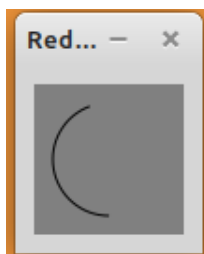
otteniamo la figura di sinistra,

```
view [base draw [arc 20x35 50x20 0 110]]
```

otteniamo la figura di centro,

```
view [base draw [arc 20x35 50x20 0 110 closed]]
```

otteniamo la figura di destra.



curve Costruisce una curva di Bezier con

`curve <inizio> <controllo> <fine>`

dove

`<inizio>` è un pair! che indica il punto di inizio della curva,

`<controllo>` è un pair! che indica il punto di controllo,

`<fine>` è un pair! che indica il punto dove finisce la curva.

Con un solo punto di controllo abbiamo una curva di Bezier quadratica. Se indichiamo due punti di controllo abbiamo una curva di Bezier cubica.

Con

```
view [base draw [curve 20x60 13x11 58x43]]
```

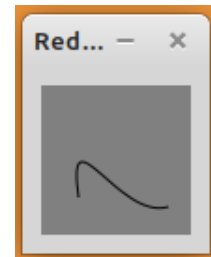
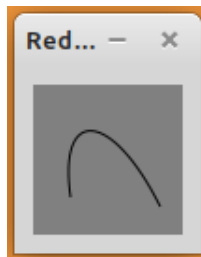
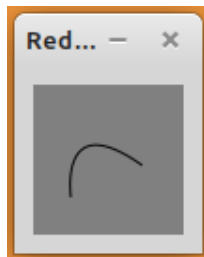
otteniamo la figura di sinistra,

```
view [base draw [curve 20x60 13x11 42x13 68x65]]
```

otteniamo la figura di centro,

```
view [base draw [curve 20x60 13x11 42x73 68x65]]
```

otteniamo la figura di destra.



spline Traccia una curva che segue una sequenza di punti con

`spline <punto> <punto> <punto> <punto> ....`

dove i punti sono indicati con altrettanti pair!.

Aggiungendo la parola `closed` dopo l'elenco dei punti la curva si chiude.

Con

```
view [base draw [spline 20x60 13x11 58x43]]
```

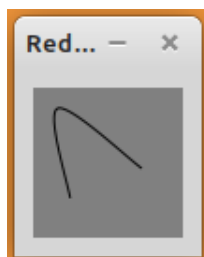
otteniamo la figura di sinistra,

```
view [base draw [spline 20x60 13x11 42x13 68x65 closed]]
```

otteniamo la figura di centro,

```
view [base draw [spline 20x60 13x11 42x73 68x65]]
```

otteniamo la figura di destra.



Ho utilizzato gli stessi punti utilizzati nel precedente paragrafo per le curve di Bezier per un utile raffronto.

A differenza di `curve`, che accetta un massimo di quattro parametri, `spline` ne accetta quanti vogliamo.

text Con

```
text <punto> <stringa>
```

inseriamo una scritta, proposta come `<stringa>` tra doppi apici, il cui angolo superiore sinistro coincide con il `<punto>`, indicato con un pair!

Con

```
view [base draw [text 20x20 "Ciao"]]
```

otteniamo



* * *

E' possibile inserire più elementi elencandoli uno di seguito all'altro.

Per esempio, con

```
view [base draw [line 10x10 70x70 text 10x10 "Cerchio" circle 40x60 15]]
```

otteniamo



Gli esempi sono stati fatti utilizzando il face base di default (80 x 80 pixel e sfondo grigio).

Per modificare questa impostazione di default basta far seguire alla parola base un pair! per modificare la dimensione e/o un nome di colore o una tupla RGB per modificare il colore.

Con

```
view [base 200x100 yellow draw [circle 50x50 30]]
```

avremo il cerchio disegnato su un piano di 200 pixel per 100 con sfondo giallo.

Anche linee e figure sono state disegnate con impostazioni di default (spessore standard minimo, colore nero, senza riempiimenti di colore per le figure).

Ora vediamo come governare queste impostazioni per ottenere disegni più avvincenti.

4.2.2 Proprietà delle linee

Tra quelli previsti dal linguaggio, penso che il più importante strumento di regolazione della proprietà delle linee, oltre a quello di dare loro un colore che vedremo nel prossimo paragrafo, sia quello di dare loro una dimensione.

Lo si può fare facendo precedere all'istruzione per disegnare la linea quest'altra istruzione `line-width <numero>`

dove <numero> è lo spessore in pixel della linea.

Per default le linee hanno lo spessore di un pixel.

Per esempio, con

```
view [base 200x100 200.200.40 draw [line-width 3 line 10x10 70x50 line-width 4 circle 120x50 45]]
```

otteniamo



4.2.3 Manipolazione delle figure

Le figure che costruiamo con le istruzioni viste nel paragrafo 14.1 sono tutte orientate in modo parallelo agli assi della lavagna in cui sono disegnate e, nel caso di box, con tutti gli angoli retti. Ciò non si nota per cerchi e archi ma per le figure squadrate e per l'orientamento dell'ellisse si.

Red ci offre due possibilità per modificare questo stato di cose: la prima ci consente di disegnare figure non parallele agli assi ma variamente inclinate rispetto ad essi (*rotate*), la seconda ci consente di disegnare figure con lati inclinati rispetto ad essi (*skew*).

rotate La sintassi è

`rotate <angolo> <centro>`

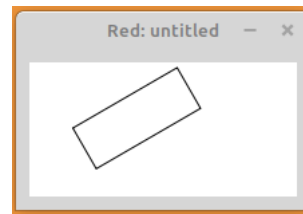
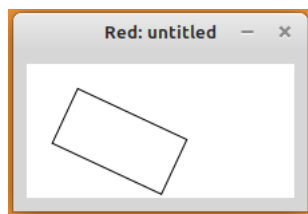
dove <angolo> è l'angolo di inclinazione della figura rispetto all'orientamento orizzontale, girando in senso orario e <centro> è il punto su cui si incentra la rotazione.

Con

```
view [base 200x100 white draw [rotate 25 20x10 box 40x10 130x55]]
```

```
view [base 200x100 white draw [rotate 25 20x10 box 40x10 130x55]]
```

otteniamo, rispettivamente, le figure di sinistra e di destra.



skew E' un comando con il quale possiamo inclinare gli assi in modo che la lavagna su cui disegniamo non sia più ortogonale, ottenendo così che le figure che vi inseriamo abbiano i lati parimenti inclinati.

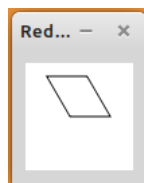
La sintassi del comando è

`skew <angolo_lungo_asse_x> <angolo_lungo_asse_y>`

Con

```
view [base white draw [skew 30 0 box 10x10 40x40]]
```

otteniamo



4.2.4 Colore

Per disegnare a colori dobbiamo procurarci una penna adatta e Red ce ne fornisce due: una per tracciare linee e una per colorare l'interno di figure.

pen E' la penna per tracciare linee e si predispone con

`pen <colore>`

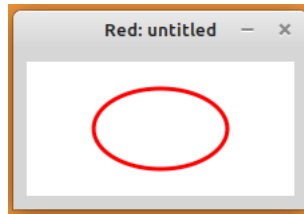
dove <colore> è il nome riconosciuto per un colore o una tupla RGB.

La predisposizione deve avvenire prima di tracciare.

Con

```
view [base 200x100 white draw [line-width 3 pen red circle 100x50 50 30]]
```

otteniamo



Per disattivare la penna si scrive `pen off`.

fill-pen E' la penna per riempire una figura e si predispone con `fill-pen <colore>`

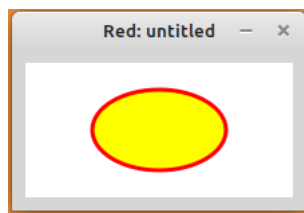
dove, al solito, `<colore>` è il nome riconosciuto per un colore o una tupla RGB.

La predisposizione deve avvenire prima di tracciare.

Con

```
view [base 200x100 white draw [line-width 3 pen red fill-pen yellow circle 100x50 50 30]]
```

otteniamo



Per disattivare la penna si scrive `fill-pen off`.

Gradiente Un gradiente di colore è una successione progressiva di tonalità cromatiche.

Red ci dà modo di arricchire le penne per tracciare linee o per riempire figure della capacità di farlo con gradienti di colori scelti da noi.

In più, con la possibilità di comporre il gradiente in forma lineare (da sinistra a destra) o in forma radiale (irradiando da un centro).

La penna viene predisposta aggiungendo alla parola `pen` o `fill-pen` la parola `linear` o la parola `radial` seguite dall'elenco dei colori da coinvolgere (indicati con nome o tupla RGB).

In particolare le sintassi sono le seguenti:

. **linear**

```
pen linear <colore> <colore> ... <inizio> <fine>
```

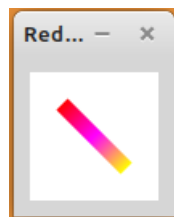
```
fill-pen linear <colore> <colore> ... <inizio> <fine>
```

Se `<inizio>` e `<fine>` non sono indicati il gradiente viene applicato a tutta la linea o figura in orizzontale da sinistra a destra.

Con

```
view [base white draw [pen linear red magenta yellow line-width 10 line 20x20 60x60]]
```

otteniamo



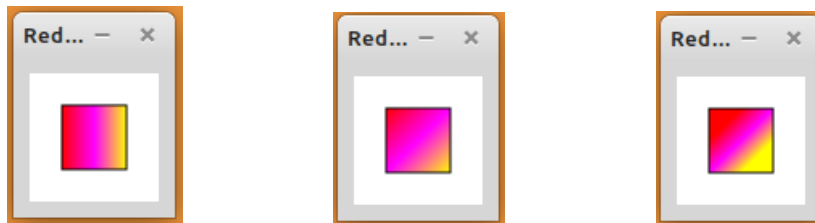
Con

```
view [base white draw [fill-pen linear red magenta yellow box 20x20 60x60]]
```

```
view [base white draw [fill-pen linear red magenta yellow 20x20 60x60 box 20x20 60x60]]
```

```
view [base white draw [fill-pen linear red magenta yellow 30x30 50x50 box 20x20 60x60]]
```

otteniamo, rispettivamente, le figure di sinistra, centro e destra.



. radial

pen linear <colore> <colore> ...<centro> <raggio>

fill-pen linear <colore> <colore> ...<centro> <raggio>

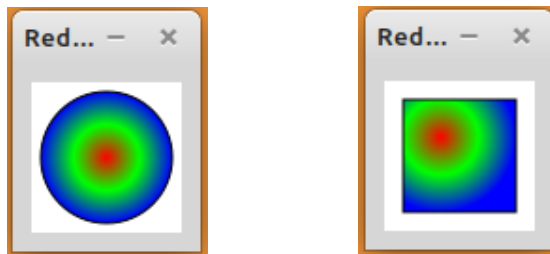
dove <centro> e <raggio>, rispettivamente da indicare con un pair! e con un numero, indicano la zona in cui operare il gradiente.

Con

```
view [base white draw [fill-pen radial red green blue 40x40 35 circle 40x40 35]]
```

```
view [base white draw [fill-pen radial red green blue 30x30 40 box 10x10 70x70]]
```

otteniamo, rispettivamente, le figure di sinistra e di destra.



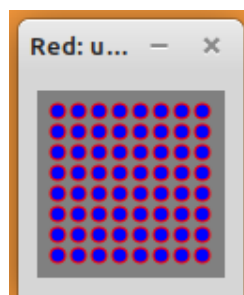
4.2.5 Disegno programmato e animazione

Ricorrendo al linguaggio Red, particolarmente ai controlli di flusso e al calcolo, possiamo scrivere programmi per produrre disegni particolarmente elaborati o animazioni.

Con questo programma

```
Red [needs: view]
disegna: [pen red fill-pen blue]
view/no-wait [foglio: base 100x100]
repeat x 8 [
  repeat y 8 [
    posizione:(x * 11x0) + (y * 0x11)
    append disegna reduce ['circle posizione 4]
  ]
]
foglio/draw: disegna
print "Fatto"
print "Scrivi quit per chiudere"
```

otteniamo



Il programma inizia con la predisposizione di un blocco per il disegno, per intanto scegliendo le penne, rossa per le linee, blu per i riempimenti.

Poi abbiamo un blocco `view` all'interno del quale, dopo aver creato un foglio per il disegno con le dimensioni giuste per contenere ciò che vogliamo disegnare, completiamo il blocco per il disegno predisponendo una griglia di posizioni corrispondenti al centro dei cerchi che vogliamo inserire nella griglia e, con la funzione `append`, inseriamo nel blocco per il disegno, valorizzandole con `reduce`, le istruzioni per il disegno dei cerchi: la parola `circle`, dovendo passare tale e quale senza valorizzazione, è preceduta da apice singolo ed è seguita dalla posizione indicante il centro dei cerchi da disegnare e dal numero indicante il raggio dei cerchi stessi.

Nell'impostare il blocco `view` abbiamo aggiunto il metodo `no-wait` per fare in modo che si avvii il ciclo per la predisposizione della griglia: senza questa aggiunta il programma si fermerebbe nel blocco `view`.

Finalmente assegniamo alla funzione `draw` del foglio per il disegno l'esecuzione del blocco per il disegno.

Dal momento che ciò avviene fuori dal blocco `view`, il disegno coinvolge la console e le istruzioni di stampa che seguono servono per tenere aperta la console stessa fino a chiusura volontaria: in mancanza di ciò la console si chiuderebbe subito dopo aver prodotto il disegno e non avremmo nemmeno il tempo per vederlo.

4.2.6 Animazione

E' noto che le animazioni si ottengono attraverso la successione rapida di figure che si modificano.

Per ottenere la successione di azioni legate al tempo Red prevede un `facet`, che si chiama `rate`, contenente un timer che pulsa a velocità regolabile e ad ogni tick genera un evento, chiamato `on-time`.

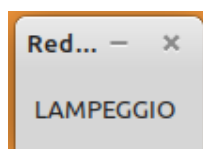
La velocità della pulsazione si ottiene aggiungendo alla parola `rate` un numero intero che indica il numero di tick al secondo.

Pertanto `rate 2` genera impulsi molto più lenti che non `rate 20`.

Per esempio, per generare una scritta lampeggiante possiamo utilizzare questo programmino

```
Red [needs: view]
view [ t: text "" rate 5
on-time [either t/text = "" [t/text: "LAMPEGGIO"] [t/text: ""]]
]
```

con cui otteniamo



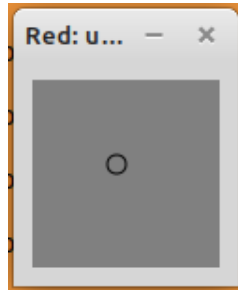
dove la scritta LAMPEGGIO scompare e ricompare 5 volte al secondo.

E' questa la base delle animazioni.

Con il seguente programmino generiamo un piccolo cerchio che attraversa la finestrella dall'angolo in alto a sinistra all'angolo in basso a destra.

```
Red [needs view]
posizione: 0x0
aggiorna-lavagna: func [] [
  posizione: posizione + 1x1
  lavagna/draw: reduce ['circle posizione 5]
]
view [
  lavagna: base 100x100 rate 25
  on-time [aggiorna-lavagna]
```

]



Cominciamo con inizializzare la posizione, che corrisponde al centro del cerchio che vogliamo far muovere, nell'angolo in alto a sinistra (coordinate rappresentate dal pair! 0x0). Poi creiamo una funzione che, ad ogni chiamata, disegna il cerchio in una posizione lievemente discosta dalla precedente (vedi posizione che si incrementa del pair! 1x1 ad ogni chiamata). Infine prevediamo una lavagna, che generalmente, nei programmi di grafica, si chiama canvas (tela), con un rate (nel caso abbastanza veloce) e richiamiamo la funzione per aggiornare la lavagna ad ogni tick con l'evento on-time.